

Apunte Física Computacional II

Integrantes: Joaquín Parra
Ignacio Falcón
Alvaro Osses
Benjamin Fuentes

Resumen

El siguiente documento presenta un apunte para el curso de Física Computacional II, basado en las notas de clase y presentaciones del ramo dictado por Dr. Roberto Navarro durante el segundo semestre de los años 2024 y 2025, y las notas de clase redactadas por Pedro Contreras.

Índice de Contenidos

1. Cálculo Numérico	1
1.1. Errores Computacionales	1
1.2. Derivadas numéricas	2
1.2.1. Otros métodos de derivación numérica	4
1.2.2. Derivadas para datos discretos	4
2. Breve introducción a Python	6
3. Introducción a la Búsqueda de Ceros	7
4. Ecuaciones Diferenciales Ordinarias	8
4.1. Método de Euler	8
4.1.1. Definición	8
4.1.1. Ejemplo	8
4.2. Método del salto de la rana	10
4.2.1. Motivación	10
4.2.1. Ejemplo	10
4.2.2. Definición: Esquemas de 2 y 3 Pasos	11
4.3. Comentarios Sobre Estabilidad*	13
4.3.1. Criterio del Jacobiano y Sistemas Autónomos	13
4.3.2. Funciones de Lyapunov	13
4.4. Métodos Runge-Kutta	13
5. Interpolación y ajustes	14
5.1. Interpolación de Lagrange	14
5.2. Matriz de Vandermonde	15
5.2.1. Casos particulares interesantes	16

Índice de Figuras

1. Error de la derivada adelantada de $f(x) = \sin(x)$ en $x = 0.3$ para distintos valores de h	3
2. Soluciones al problema del resorte colgando. Note como $y_{eq} = -9.8(g/\omega^2)$ pues en ese punto $y'' = 0$ y y' es máxima.	10
3. Gráficos de energía y espacio de fases para el método de Euler.	11
4. Conjunto de puntos interpolados por Lagrange y función original.	15

Índice de Códigos

1.	Suma de dos decimales en Python	1
2.	Output	1
3.	Algoritmo básico del método de Euler.	9
4.	Algoritmo básico del método leap-frog.	12
5.	Esquema de 3 pasos para el salto de rana.	13

1. Cálculo Numérico

1.1. Errores Computacionales

Si usted ha intentado realizar una suma de dos números decimales mediante algún lenguaje de programación, podrá haber notado que el resultado no es exactamente correcto

Código 1: Suma de dos decimales en Python

```
1 a = 0.1
2 b = 0.2
3 c = 0.3
4
5 print(f"{a + b == c}")
6 print(f"{a + b}")
```

Cuyo output será

Código 2: Output

```
1 False
2 0.30000000000000004
```

Este error se debe a la manera en que las computadoras deben representar números. En general, los humanos representamos los números en base 10, por ejemplo, escribimos el 325 como

$$325 = 3 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0 = [325]_{10}$$

Pero los computadores utilizan bits (0 y 1), por lo que estos representan números en base 2, es decir:

$$325 = 2^8 + 2^6 + 2^2 + 2^0 = [101000101]_2$$

Ahora, para representar número reales en el sistema binario, se deben incluir exponentes negativos a la suma de exponentes. En la representación binaria, los números se separan por un exponente y una parte fraccionaria (mantisa) las cuales son separadas por un punto (.). Por ejemplo,

$$\begin{aligned} [2.145]_{10} &= 2 \cdot 10^0 + \frac{1}{8} \\ &= 2^1 + 2^{-3} \\ &= [10.001]_2 \\ &= [1.0001]_2 \cdot 2^1 \end{aligned}$$

Este ejemplo en particular funciona bien porque la mantisa del número decimal resulta ser un exponente de 2, veamos que ocurre para un número más complejo:

$$\begin{aligned}
[0.1]_{10} &= [1]_2/[1010]_2 \\
&= [0.00011]_2 \\
&= [0.00011001100110011 \dots] \\
&= 2^{-4} + 2^{-5} + 2^{-8} + 2^{-9} + 2^{-12} + 2^{-13} + \dots \\
&= 2^{-4} \cdot (1 + 2^{-1} + 2^{-4} + 2^{-5} + \dots)
\end{aligned}$$

Obtenemos una serie infinita de potencias de dos. Como en la practica los computadores tienen memoria finita es necesario truncar la serie. En general, la mantisa recibe 52 bits, por lo que la serie se trunca hasta el termino 2^{-52} . en efecto:

$$[0.1]_{10} = 2^{-4} \cdot (1 + 2^{-1} + 2^{-4} + 2^{-5} + \dots + 2^{-52})$$

Este número no será *exactamente* igual a 0.1, si no que será igual a $0.1 \cdot (1 + \epsilon)$. Este será el caso para la gran mayoría de los calculos hechos computacionalmente, el resultado final tendrá una pequeña divergencia con respecto al valor real, es decir

$$\bar{a} = a \cdot (1 + \epsilon)$$

donde $\bar{a}$ y a son los valores calculado y real, respectivamente. La diferencia entre el número deseado y el resultante se denomina **error computacional**. Dado que las computadores no pueden físicamente lidiar con cantidades infinitesimalmente pequeñas, muchos de los métodos de cálculo adaptados a la programación que veremos a lo largo del apunte serán basados en aproximaciones, por esta razón es importante aprender a identificar fuentes de errores numericos y sus ordenes de magnitud.

1.2. Derivadas numéricas

Sabemos del calculo diferencial que la derivada de una función arbitraria f evaluada en un punto arbitrario x_0 es dada por

$$f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h} \quad (1)$$

pero ya sabemos que necesitamos que h sea un número finito, por lo que estamos obligados a seguir anaizando esta definición. De la serie de taylor de f evaluada en un punto $a + h$, $h > 0$, tenemos que:

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(a)h^2 + \frac{1}{3!}f'''(a)h^3 + \dots \quad (2)$$

Podemos reemplazar los terminos de orden superior por un $f(\xi)h^2$, $a \leq \xi \leq a + h$ (Teorema del valor medio), luego

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(\xi)h^2 \quad (3)$$

$$f'(a) = \frac{f(a + h) - f(a)}{h} - \frac{1}{2!}f''(\xi)h \quad (4)$$

Aquí, hemos obtenido una expresión finita para la derivada de una función evaluada en un punto, denominada *derivada adelantada*. Ahora, si buscamos el error de la ecuación (4) vemos que

$$Error = \left| f'(a) - \frac{f(a + h) - f(a)}{h} \right| = \frac{1}{2!} |f''(\xi)| h \quad (5)$$

en notación $\mathcal{O}$ de Landau,

$$Error = \frac{1}{2!} |f''(\xi)| h = \mathcal{O}(h) \quad (6)$$

Esto quiere decir que el error de truncamiento de la derivada adelantada será del orden de h , como se puede observar en la figura 1, el error de la derivada decrece linealmente con el orden de magnitud de h hasta aproximadamente $h \sim 10^{-8}$, pasado ese punto h se vuelve demasiado pequeño para la computadora y predomina el error computacional. Por esta razón, a la hora de implementar este, o cualquiera de los métodos expuestos en este apunte, es conveniente utilizar un $h \sim 10^{-5}$.

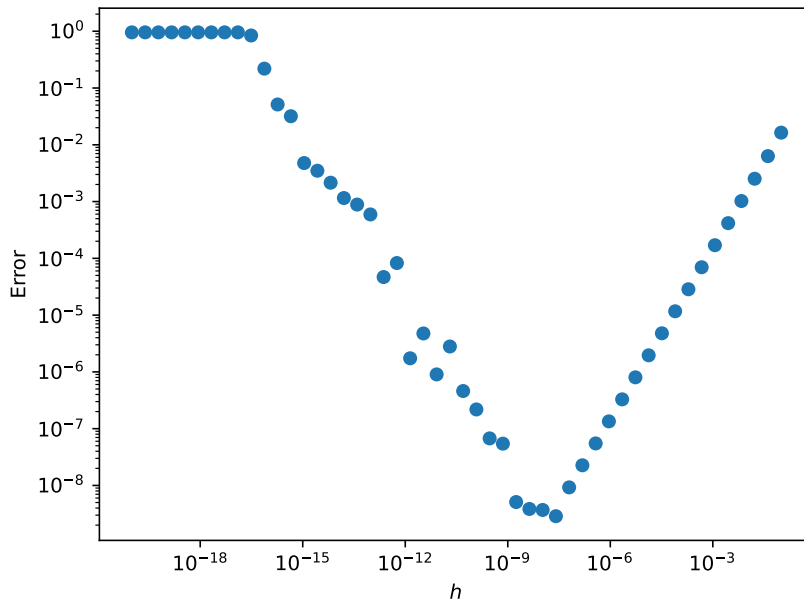


Figura 1: Error de la derivada adelantada de $f(x) = \sin(x)$ en $x = 0.3$ para distintos valores de h

1.2.1. Otros métodos de derivación numérica

Si bien la derivada adelantada es útil debido a lo simple de su implementación, muchas veces queremos utilizar métodos cuyo error sea de un mayor orden de h , es decir, menor.

Similarmente a como obtuvimos la expresión (4), podemos obtener una derivada **retrasada**, en efecto, consideramos la serie de taylor de $f(a - h)$,

$$f(a - h) = f(a) - f'(a)h + \frac{1}{2!}f''(\xi)h^2 \quad (7)$$

$$f'(a) = \frac{f(a) - f(a - h)}{h} + \frac{1}{2!}f''(\xi)h \quad (8)$$

Si consideramos las series de taylor de $f(a + h)$ y $f(a - h)$ expandidas hasta el tercer orden,

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(a)h^2 + \frac{1}{3!}f'''(\xi)h^3 \quad (9)$$

$$f(a - h) = f(a) - f'(a)h + \frac{1}{2!}f''(a)h^2 - \frac{1}{3!}f'''(\xi)h^3 \quad (10)$$

Ahora si restamos obtenemos,

$$f(a + h) - f(a - h) = 2f'(a)h + \frac{2}{3!}f'''(\xi)h^3 \quad (11)$$

$$2f'(a)h = f(a + h) - f(a - h) - \frac{2}{3!}f'''(\xi)h^3 \quad (12)$$

$$f'(a) = \frac{f(a + h) - f(a - h)}{2h} - \frac{1}{6}f'''(\xi)h^3 \quad (13)$$

Así, obtenemos una expresión para la **derivada centrada**, con error del orden de h^3 .

Podemos obtener muchos otros métodos de derivación numérica por medios similares, en general el proceso consiste en jugar con series de taylor hasta poder despejar el termino $f'(a)$.

1.2.2. Derivadas para datos discretos

En general, uno no suele tratar con funciones definidas analíticamente, si no que tendremos conjuntos de datos relacionados a alguna magnitud física, si quisieramos eventualmente encontrar la derivada de la "función" que definen estos datos, debemos aprender a tratar con puntos no necesariamente equidistantes.

Similarmente a caso anterior, dado un conjunto de puntos x_k y una función continua y diferenciable tal que $f(x_k) = f_k$, definimos la distancia entre dos puntos como $h_k = x_k - x_{k-1}$. Podemos obtener una primera aproximación simplemente tomando la pendiente entre dos puntos, esto es

$$\frac{df_k}{dx_k} \approx \frac{f_k - f_{k-1}}{x_k - x_{k-1}} \quad (14)$$

Esto es analogo a la derivada atrasada encontrada en la sección anterior, y se puede demostrar

que tendrá un error del orden de h_k . Notar además que si imponemos que los nodos estén igualmente espaciados, es decir, $x_k - x_{k-1} = h = cte.$, la ecuación (14) recupera la ecuación (8).

Ahora, estamos incentivados a encontrar un método de derivación discreta con error de menor orden, para ello consideramos la serie de Taylor de f_{k+1} y f_{k-1}

$$f_{k+1} = f_k + \frac{df_k}{dx}h_{k+1} + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_{k+1}^2 + \frac{1}{3!}\frac{d^3f(\xi_1)}{dx^3}h_{k+1}^3 \quad (15)$$

$$f_{k-1} = f_k - \frac{df_k}{dx}h_k + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_k^2 - \frac{1}{3!}\frac{d^3f(\xi_2)}{dx^3}h_k^3 \quad (16)$$

con $x_{k-1} \leq \xi_2 \leq x_k \leq \xi_1 \leq x_{k+1}$. Despejando $\frac{df_k}{dx}$ en ambas expresiones obtenemos,

$$\frac{df_k}{dx}h_{k+1} = f_{k+1} - f_k - \frac{1}{2!}\frac{d^2f_k}{dx^2}h_{k+1}^2 - \frac{d^3f(\xi_1)}{dx^3}h_{k+1}^3 \quad (17)$$

$$\frac{df_k}{dx}h_k = f_k - f_{k-1} + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_k^2 + \frac{d^3f(\xi_2)}{dx^3}h_k^3 \quad (18)$$

Multiplicando (17) y (18) por h_k/h_{k+1} y h_{k+1}/h_k , respectivamente,

$$\frac{df_k}{dx}h_k = \frac{(f_{k+1} - f_k)h_k}{h_{k+1}} - \frac{1}{2!}\frac{d^2f_k}{dx^2}h_kh_{k+1} - \frac{d^3f(\xi_1)}{dx^3}h_{k+1}^2h_k \quad (19)$$

$$\frac{df_k}{dx}h_{k+1} = \frac{(f_k - f_{k-1})h_{k+1}}{h_k} + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_kh_{k+1} + \frac{d^3f(\xi_2)}{dx^3}h_k^2h_{k+1} \quad (20)$$

Sumando,

$$[h_{k+1} + h_k]\frac{df_k}{dx} = \frac{(f_{k+1} - f_k)h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})h_{k+1}}{h_k} + \frac{1}{3!}[f'''(\xi_2)h_k - f'''(\xi_1)h_{k+1}]h_kh_{k+1} \quad (21)$$

Finalmente,

$$\frac{df_k}{dx} = \frac{(f_{k+1} - f_k)}{h_{k+1} + h_k} \frac{h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})}{h_{k+1} + h_k} \frac{h_{k+1}}{h_k} + \frac{1}{3!} \left[\frac{f'''(\xi_2)h_k - f'''(\xi_1)h_{k+1}}{h_{k+1} + h_k} \right] h_kh_{k+1} \quad (22)$$

Ahora, por el teorema del valor medio, podemos reemplazar el último termino entre corchetes por una función evaluada en un punto ξ , con $\xi_2 \leq \xi \leq \xi_1$, en efecto,

$$\frac{df_k}{dx} = \frac{(f_{k+1} - f_k)}{h_{k+1} + h_k} \frac{h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})}{h_{k+1} + h_k} \frac{h_{k+1}}{h_k} + \frac{1}{3!} \frac{d^3f_k(\xi)}{dx^3} h_kh_{k+1} \quad (23)$$

$$= \frac{(f_{k+1} - f_k)}{h_{k+1} + h_k} \frac{h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})}{h_{k+1} + h_k} \frac{h_{k+1}}{h_k} + \mathcal{O}(h_kh_{k+1}) \quad (24)$$

Así, logramos encontrar una expresión para la derivada de una función definida punto

a punto con nodos no necesariamente equidistantes, con error de orden cuadrático. Puede comprobar que si imponemos que los nodos sean equidistantes ($h_k = h_{k+1} = h$), se recupera la expresión (13) para la derivada centrada (**Demuestrelo!**). Más adelante estudiaremos métodos más avanzados para tratar con distribuciones de datos con interpolaciones y splines. Para terminar, cabe mencionar que la mayor desventaja de este método es que, al utilizar 3 puntos, aplicándola a una serie de puntos, la expresión numérica de la derivada tendrá dos puntos menos que la serie original. Por esta razón, en la práctica se implementa una derivada adelantada para los puntos iniciales de un set de datos, y una atrasada para los finales; la derivada centrada se utiliza solo para los puntos intermedios.

2. Breve introducción a Python

Hola

3. Introducción a la Búsqueda de Ceros

Usualmente, cuando trabajamos con ecuaciones, una de nuestras metas es encontrar el valor de cierta variable o también llamada incógnita, para esto, desarrollamos una serie de pasos o "algoritmo" que seguir para encontrar la respuesta. Un buen ejemplo de esto es considerar el caso de una ecuación cuadrática:

$$ax^2 + bx + c = 0 \quad (25)$$

Es bien sabido que, dado los coeficientes a , b y c , existe una fórmula general que seguir, la conocida como ecuación cuadrática o *Ecuación de Bashkara*:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (26)$$

Al introducir los respectivos valores de los coeficientes, podemos encontrar las dos soluciones a este problema.

Sin embargo, no siempre se puede encontrar una ecuación para los valores de incógnitas en toda ecuación, un ejemplo a esto es el siguiente:

$$\sin x + x = 0 \quad (27)$$

A pesar de lo simple de nuestro problema (puesto que es la suma entre una función trigonométrica y la incógnita), ya no es posible encontrar una ecuación de forma analítica que exprese valores exactos de las soluciones para " x ", así, la pregunta es ¿Qué podemos hacer en estos casos?

Para esto, se desarrollan los **Métodos para hallar ceros de una función**

meow

4. Ecuaciones Diferenciales Ordinarias

4.1. Método de Euler

4.1.1. Definición

El método de Euler es el acercamiento más simple a la resolución numérica de ecuaciones diferenciales ordinarias. Consiste en realizar una simple aproximación en Taylor a primer orden. Dada la simplicidad del método, se abordarán de inmediato los casos en una y varias dimensiones.

Considere que busca resolver una ecuación diferencial ordinaria de orden n . Como recordará de su curso de ecuaciones diferenciales ordinarias, podemos representar este problema como una EDO de primer orden en forma vectorial:

$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_n \end{pmatrix} = \frac{d\vec{x}}{dt} = \vec{f}(\vec{x}, t), \quad (28)$$

con $\vec{x} = (x_1(t), x_2(t), \dots, x_n(t))$ un vector de más funciones a encontrar y $\vec{f}$ una función vectorial conocida. En su curso de EDO quizás vio esta formulación con la forma:

$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_{n-1} \\ \dot{x}_n \end{pmatrix} = \begin{pmatrix} x_2 \\ x_3 \\ \vdots \\ x_n \\ g(x) \end{pmatrix}, \quad (29)$$

con $g(x)$ generalmente la propia EDO en cuestión. Esta formulación es sumamente útil y será utilizada más adelante. Aproximando (28) como una derivada adelantada, tenemos el **método de Euler**:

$$\vec{x}(t+h) = \vec{x}(t) + h\vec{f}(\vec{x}, t) + \mathcal{O}(h^2). \quad (30)$$

Esto define un método de primer orden. En general, el orden n de un integrador o método se suele expresar como el n tal que los pasos tienen error de orden $\mathcal{O}(h^{n+1})$. Geométricamente, lo que sucede paso a paso es que partimos situados en un punto $\vec{x}_0$ dado por una condición inicial en un tiempo t_0 , y avanzamos al siguiente punto siguiendo el vector tangente $f(\vec{x}_0, t)$. Abordaremos el algoritmo con un problema.

Ejemplo

Consideremos un objeto de masa m colgado de un resorte de constante elástica k , en un campo gravitatorio de magnitud g . Si el origen del sistema de referencia se encuentra ubicado

en el resorte en su longitud natural, entonces la altura $y(t)$ se rige por la ecuación diferencial:

$$y''(t) + \omega^2 y(t) = -g, \quad (31)$$

donde $\omega = \sqrt{k/m}$. Si tenemos las condiciones iniciales $y(0), y'(0)$, la solución analítica es:

$$y(t) = \left[y(0) + \frac{g}{\omega^2} \right] \cos(\omega t) + \frac{y'(0)}{\omega} \sin(\omega t) - \frac{g}{\omega^2}. \quad (32)$$

Como la ecuación diferencial es de segundo orden, podemos expresarla en forma vectorial para reducir el problema a una ecuación diferencial de primer orden. Tenemos:

$$\frac{dy}{dt} = v(t), \quad \frac{dv}{dt} = -\omega^2 y(t) - g, \quad (33)$$

es decir, podemos reescribir el problema como:

$$\vec{x} = \begin{pmatrix} y(t) \\ v(t) \end{pmatrix}, \quad \vec{f}(\vec{x}, t) = \begin{pmatrix} v(t) \\ -\omega^2 y(t) - g \end{pmatrix}, \quad \frac{d\vec{x}}{dt} = \vec{f}(\vec{x}, t). \quad (34)$$

De esta forma, el algoritmo relevante del método es¹:

Código 3: Algoritmo básico del método de Euler.

```

1  def f(x, omega=1.0, g = 9.8): # creamos el vector f, acá hay que definir omega
2      y, v = x # desempaquetamos las variables
3
4      return array([v, -y * omega ** 2 - g])
5
6  x = zeros([Ntiempo, 2]) # Definimos un vector x con los pares (x,v) a cada paso
7  x[0] = [y0, v0] # Imponemos que la primera entrada de x sean las cond. iniciales
8
9
10 for n in range (Ntiempo):
11     x[n+1] = x[n] + h * f(x[n], omega, g) #partiendo de x[0], cambia los pares (0,0) por los
        ↪ valores del método
12 #finalmente, solo hace falta desempaquetar el resultado del vector x.
```

Finalmente, para ver la eficacia del método, se ha simulado tanto la solución analítica como la solución mediante el método de Euler en la figura 2. Podemos apreciar

¹ Este código es un esquema. Para ver código real, puede visitar **Link del repo cuando esté terminado**

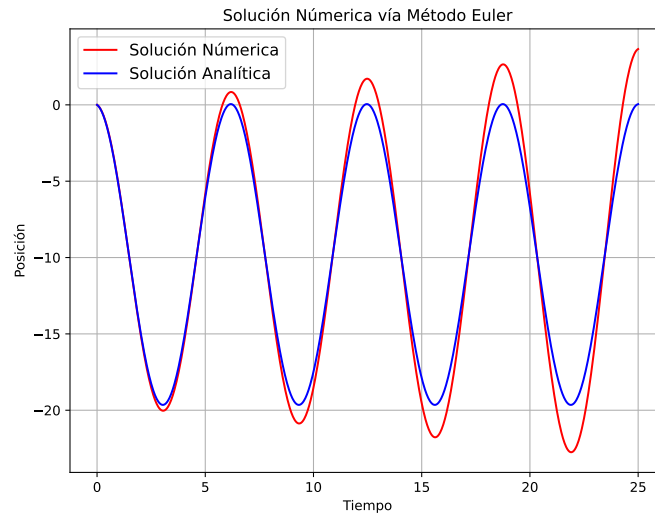


Figura 2: Soluciones al problema del resorte colgando. Note como $y_{eq} = -9.8(g/\omega^2)$ pues en ese punto $y'' = 0$ y y' es máxima.

4.2. Método del salto de la rana

4.2.1. Motivación

Si bien el método de Euler es un approach simple y familiar (pero no muy estable) a la resolución numérica de EDO's, **físicamente** plantea un problema importante: **no conserva la energía del sistema**. Un rápido cálculo nos muestra esto:

Ejemplo

Para un sistema masa-resorte colgando, se puede mostrar que **energía total** del sistema es de la forma:

$$E(t) = \frac{1}{2}mv^2 + \frac{1}{2}m^2\omega^2 \left(y + \frac{g}{\omega^2} \right)^2. \quad (35)$$

Luego, bajo el método de Euler, la energía en tiempos discretos es:

$$\begin{aligned} E_{n+1} &= \frac{1}{2}mv_{n+1}^2 + \frac{1}{2}m^2\omega^2 \left(y_{n+1} + \frac{g}{\omega^2} \right)^2, \\ &= \frac{1}{2}m \left[v_n - h(\omega^2 y_n + g) \right]^2 + \frac{1}{2}m^2\omega^2 \left[y_n + hv_n + \frac{g}{\omega^2} \right]^2, \\ &= (1 + \omega^2 h^2) \left[\frac{1}{2}mv_n^2 + \frac{1}{2}m^2\omega^2 \left(y_n + \frac{g}{\omega^2} \right)^2 \right], \\ &= (1 + \omega^2 h^2) E_n. \end{aligned}$$

Es decir, el método de Euler no conserva la energía pues $E_{n+1} > E_n$ para todo n . Puede ver esto también modelando la ecuación (35), o realizando un gráfico de rapidez versus posición (una especie de espacio de fases) mientras evoluciona el tiempo (ver figura 3). El hecho que la trayectoria no sea cerrada en el espacio de fases es consecuencia directa de la pérdida de energía.

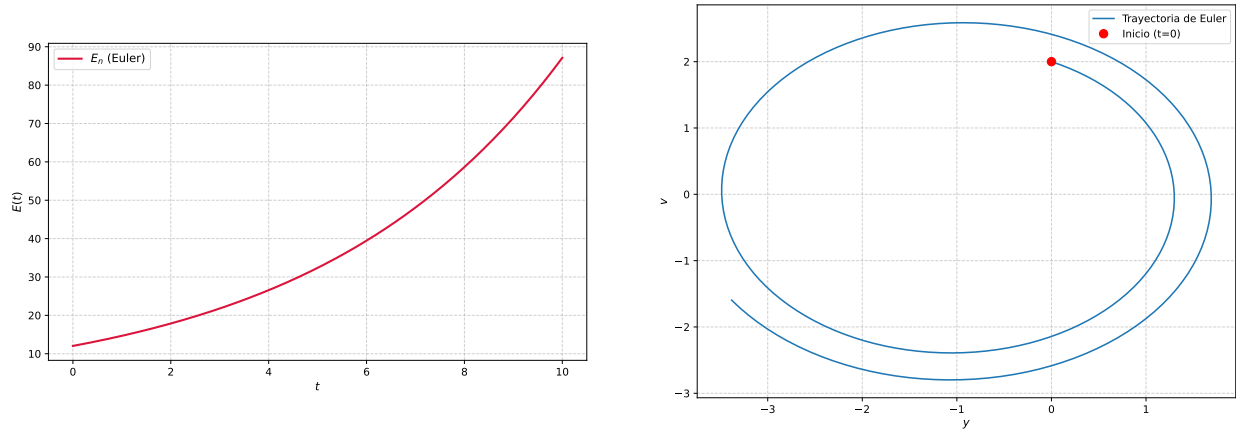


Figura 3: Gráficos de energía y espacio de fases para el método de Euler.

4.2.2. Definición: Esquemas de 2 y 3 Pasos

Este método resulta sumamente útil al momento de lidiar con sistemas en los cuales hay **conservación de la energía mecánica** puesto que su formulación basada en pasos semi-enteros incorpora una estructura simpléctica² (conserva la energía). Este tipo de métodos suelen ser llamados **integradores simplécticos**. El método en cuestión consiste en usar un esquema de derivadas centradas que permiten que, para un paso Δt , se actualice la información de la rapidez dos veces por paso. Una primera formulación del método consiste en definir:

$$v(t + h/2) = \frac{x(t + h) - x(t)}{h} + \mathcal{O}(h^2), \quad (36)$$

$$a(t) = \frac{v(t + h/2) - v(t - h/2)}{h} + \mathcal{O}(h^2), \quad (37)$$

lo que define un integrador de segundo orden con 2 pasos. Multiplicando ambos lados por h , tenemos :

$$v(t + h/2) = v(t - h/2) + ha(t) + \mathcal{O}(h^3), \quad (38)$$

$$x(t + h) = x(t) + hv(t + h/2) + \mathcal{O}(h^3). \quad (39)$$

² La demostración de esto es muy compleja. Para más acerca de integradores simplécticos, puede leer este [artículo](#), que puede encontrar [aquí](#).

Note que el método requiere una condición inicial $v(t_0) = v_0$ a partir del cual computamos los $v_{n+1/2}$ (esto siempre ocurrirá en esquemas basados en derivadas centradas) además de tener información acerca de la aceleración de antemano. Por esto, el esquema suele iniciarse con el método de Euler³:

$$v(h/2) = v(0) + \frac{h}{2}a(0).$$

Bajo estos supuestos, el algoritmo relevante que utiliza el método es

Código 4: Algoritmo básico del método leap-frog.

```

1  # Condiciones iniciales
2  x[0], v[0] = x0, v0
3  # Metodo de Euler para avanzar a t = h/2
4  # Escribimos v_1/2 = v_0 + (h/2)a_0
5  v[0] = v[0] + (h/2) * a[0] # v[0] representa a v_1/2
6
7  # Metodo del salto de rana
8  for n in range(N):
9      x[n+1] = x[n] + h * v[n] #v[n] representa v_n+1/2
10     v[n+1] = v[n] + h * a[n] #v[n+1] representa v_n+3/2
11     ventero[n+1] = 0.5 * (v[n+1] + v[n]) # calcula v_n
12

```

Notemos que bajo este esquema, no tenemos información acerca de la velocidad en tiempos enteros, por lo que generalmente se suele considerar la media aritmética entre 2 velocidades adyacentes (esto se ha implementado en la última línea).

También es útil implementar este método como un esquema de 3 pasos, calculando velocidades tanto a tiempos enteros como semi-enteros. La utilidad de esta formulación es que dejamos de depender del método de Euler para inicializar $v_{1/2}$. A continuación, se muestra el razonamiento para componer el esquema. Consideremos la derivada centrada en $t + h/2$:

$$v(t + h/2) = \frac{x(t + h) - x(t)}{2 \cdot h/2} + \mathcal{O}(h^2), \quad (40)$$

$$\Rightarrow x(t + h) = x(t) + hv(t + h/2) + \mathcal{O}(h^3). \quad (41)$$

Ahora, para la velocidad, consideremos la derivada centrada en t de v :

$$a(t) = v'(t) = \frac{v(t + h/2) - v(t - h/2)}{h} + \mathcal{O}(h^2) \quad (42)$$

$$\Rightarrow v(t + h/2) = v(t - h/2) + ha(t) + \mathcal{O}(h^3). \quad (43)$$

³ Esto no significa que todo el método sea tipo Euler.

Expandiendo $v(t - h/2)$ alrededor de t , tenemos:

$$v(t + h/2) = v(t) - \frac{h}{2}v'(t) + ha(t) + \mathcal{O}(h^3), \quad (44)$$

$$\Rightarrow v(t + h/2) = v(t) + \frac{h}{2}a(t) + \mathcal{O}(h^3). \quad (45)$$

Finalmente, para obtener el término $v(t + h)$ (el paso entero), se considera una expansión a primer orden centrada en $t + h/2$:

$$v(t + h) = v(t + h/2) + \frac{h}{2}a(t + h/2). \quad (46)$$

Las ecuaciones (41), (45) y (46) definen el esquema de 3 pasos del método. En forma de código, el algoritmo relevante es:

Código 5: Esquema de 3 pasos para el salto de rana.

```

1  # condiciones iniciales
2  x[0], v[0] = x0, v0
3
4  # Metodo de salto de rana
5  for n in range(N-1):
6      vmedio = v[n] + (h/2) * a[n] # v[n] representa v_n
7      x[n+1] = x[n] + h * vmedio # vmedio representa v_{n+1/2}
8      v[n+1] = vmedio + (h/2) * a[n+1] # v[n+1] representa v_{n+1}
9

```

Generalmente, al momento de implementar el método, el problema se suele abordar de manera vectorial, y uno resuelve el sistema:

$$\begin{pmatrix} \dot{x} \\ \dot{v} \end{pmatrix} = \begin{pmatrix} v \\ a \end{pmatrix} \quad (47)$$

Este tipo de formulación se volverá común una vez que vea Mecánica Clásica, en particular, el formalismo simpléctico en la teoría Hamilton-Jacobi. Un problema completo se abordará más adelante.

ACÁ FALTA CÓDIGO REAL EN FORMA VECTORIAL.

4.3. Comentarios Sobre Estabilidad*

4.3.1. Criterio del Jacobiano y Sistemas Autónomos

4.3.2. Funciones de Lyapunov

4.4. Métodos Runge-Kutta

5. Interpolación y ajustes

Como mencionamos muy anteriormente, muchas veces no tendremos el lujo de lidiar con funciones definidas analíticamente en todos sus puntos, en lugar de eso, tendremos algún conjunto discreto de puntos $(x_i, f_i)_{i=0, \dots, N}$. En estos casos podríamos querer hacer una de dos cosas: interpolar, o bien, ajustar.

5.1. Interpolación de Lagrange

La interpolación consiste en encontrar una función que pase por todos y cada uno de los puntos dados. En estricto rigor estamos llenando el espacio entre medio de los puntos. Si le suena familiar, es porque esto es lo que hacen los modelos de lenguaje (Gemini, ChatGPT, Claude, etc); la diferencia es que estos lo hacen con sets de datos enormes. Ahora, dados un conjunto arbitrario de puntos $(x_i, f_i)_{i=0, \dots, N}$ siempre podemos encontrar un polinomio de grado menor o igual a N que interpole los puntos, es decir

$$f_i = p(x_i) \quad (48)$$

Podemos proponer una combinación lineal de polinomios de la forma

$$L(x) = \sum_{j=0}^N f_j \ell_j(x) \quad (49)$$

imponiendo la condición

$$\ell_j(x_i) = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases} \quad (50)$$

con tal que se cumpla (48). Podemos encontrar que los polinomios $\ell_j(x)$, denominados **polinmios base de Lagrange**, son dados por:

$$\ell_j(x) = \left(\frac{x - x_0}{x_j - x_0} \right) \left(\frac{x - x_1}{x_j - x_1} \right) \dots \left(\frac{x - x_{j-1}}{x_j - x_{j-1}} \right) \left(\frac{x - x_{j+1}}{x_j - x_{j+1}} \right) \dots \left(\frac{x - x_N}{x_j - x_N} \right) \quad (51)$$

$$= \prod_{i=0, i \neq j}^N \left(\frac{x - x_i}{x_j - x_i} \right) \quad (52)$$

y forman (49), denominado **polinomio interpolador de Lagrange**.

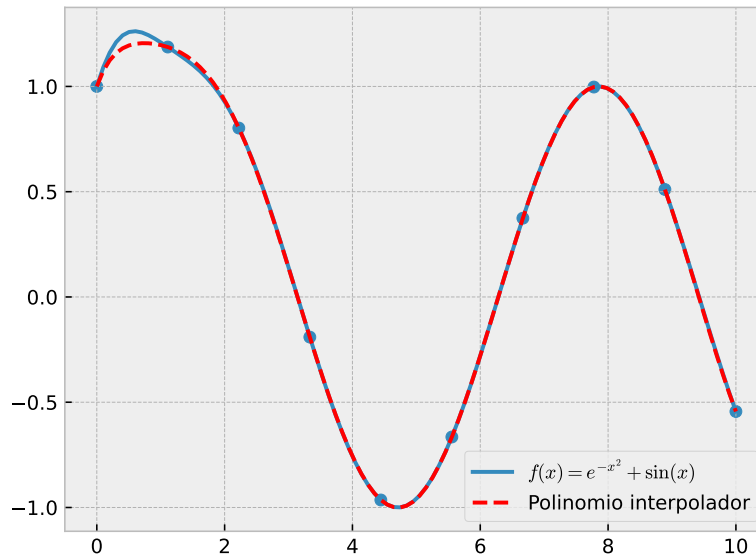


Figura 4: Conjunto de puntos interpolados por Lagrange y función original.

5.2. Matriz de Vandermonde

En la sección anterior, logramos interpolar un conjunto de puntos utilizando una combinación lineal de polinomios (vea (49)), por lo que es natural preguntarse si existe algún otro tipo de función interpoladora. La respuesta es que podemos interpolar con cualquier tipo de función que forme una **base del espacio de funciones**. Consideremos que queremos interpolar un conjunto de puntos (x_j, f_j) por medio de una función $p(x)$ tal que

$$f_j = p(x_j)$$

tenemos que

$$p(x) = \sum_{i=0}^N p_i \varphi_i(x) \quad (53)$$

$$= p_0 \varphi_0(x) + p_1 \varphi_1(x) + \cdots + p_N \varphi_N(x) \quad (54)$$

Así, tenemos el sistema de ecuaciones

$$f_j = p_0 \varphi_0(x_j) + p_1 \varphi_1(x_j) + \cdots + p_N \varphi_N(x_j) \quad (55)$$

donde queremos encontrar los p_j . Matricialmente tenemos que

$$\begin{bmatrix} \varphi_0(x_0) & \varphi_1(x_0) & \cdots & \varphi_N(x_0) \\ \varphi_0(x_1) & \varphi_1(x_1) & \cdots & \varphi_N(x_1) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_0(x_N) & \varphi_1(x_N) & \cdots & \varphi_N(x_N) \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_N \end{bmatrix} = \begin{bmatrix} f_0 \\ f_1 \\ \vdots \\ f_N \end{bmatrix} \quad (56)$$

De esta manera, reducimos el problema a resolver el sistema encontrando la inversa de la matriz $\varphi_i(x_j)$. Finalmente, todo este proceso es una generalización de encontrar los coeficientes de una serie que aproxime una función, y es analogo a encontrar los coeficientes de una serie de Taylor. El tipo de serie por el cual estemos interpolando depende de como elijamos la función $\varphi_n(x)$, por ejemplo

- Serie de Fourier: $\varphi_n(x) = \exp^{in\pi x/L}$
- Serie de polinomios de Lagrange: $\varphi_n(x) = \ell_n(x)$
- Serie de Legendre: $\varphi_n(x) = P_n(x)$ (Polinomios de Legendre)

5.2.1. Casos particulares interesantes

Si elegimos los polinomios de grado n como nuestra base de funciones, es decir, $\varphi_n(x) = x^n$, la matriz del lado izquierdo de (56) se convierte en la matriz (57)

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^N \\ 1 & x_1 & x_1^2 & \cdots & x_1^N \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^N \end{bmatrix} \quad (57)$$

denominada **Matriz de Vandermonde**.

Alternativamente, si hacemos que $\varphi_n(x) = \ell_n(x)$, por la propiedad (50), la matriz izquierda de (56) se convierte en la matriz identidad de $N \times N$, lo que reafirma la condición (48).